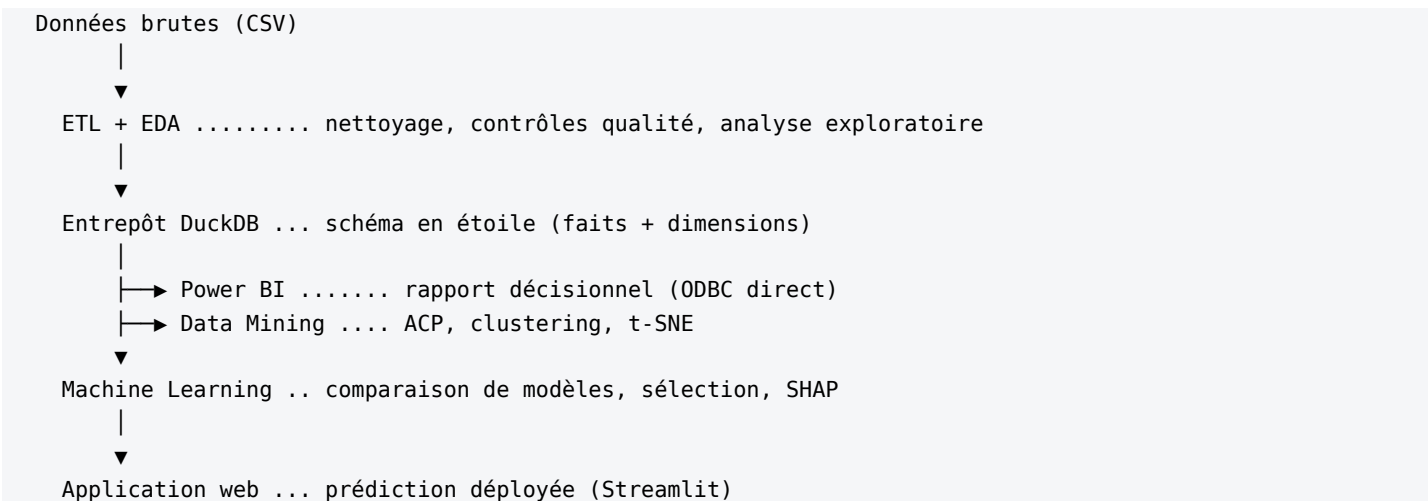


# Démarche méthodologique

Ce document décrit l'approche suivie module par module, et les principes transversaux qui garantissent la cohérence et la reproductibilité de l'ensemble.

## Vue d'ensemble du pipeline



## Déroulé par module

### 1. ETL et analyse exploratoire

Extraction du CSV, harmonisation des types, création de variables dérivées (classe d'IMC selon l'OMS, score de facteurs de risque, jours de mal-être cumulés), et contrôles qualité bloquants avant chargement. L'analyse exploratoire caractérise la cible, les distributions, la prévalence par segment, et établit les contraintes que l'exploration impose à la modélisation.

**Principe clé** : aucune donnée n'est supprimée ni imputée. Les anomalies sont signalées par des drapeaux ; les décisions de traitement appartiennent aux étapes aval, prises en connaissance de cause.

### 2. Entrepôt de données

Construction d'un schéma en étoile sous DuckDB : une table de faits (un répondant par ligne) entourée de sept dimensions reconstruites depuis le codebook. Les clés de dimension sont les codes source eux-mêmes (déjà de petits entiers contigus). L'intégrité référentielle est vérifiée à chaque construction.

**Décision documentée** : pas de dimension temporelle. Les données étant transversales (un instantané en 2015), une dimension temps serait dégénérée et sans apport analytique.

### 3. Power BI

Rapport décisionnel de six pages (deux descriptives, quatre analytiques), branché en **ODBC direct** sur l'entrepôt DuckDB en lecture seule. Le nommage cohérent des clés permet le câblage automatique des relations. Les mesures sont écrites en DAX.

### 4. Machine Learning

Comparaison de plusieurs familles de modèles (régression logistique, kNN, arbre de décision, SVM, forêt aléatoire, XGBoost). Le préprocessing (imputation, mise à l'échelle, encodage) est intégré à un pipeline **ajusté sur le seul jeu d'entraînement** pour éviter toute fuite. Sélection par ROC-AUC, avec report du PR-AUC, du rappel et du F1. Interprétation par SHAP.

### 5. Data Mining

Exploration non supervisée, sans utiliser la cible : ACP (réduction de dimension), k-means et classification ascendante hiérarchique (profils de population), t-SNE (visualisation). Le nombre de clusters est choisi par la méthode du coude, la silhouette et — surtout — l'interprétabilité des groupes obtenus.

## 6. Application web

Application Streamlit réutilisant le pipeline entraîné : formulaire de prédiction individuelle, scoring par lot, indicateurs clés. Déployée pour obtenir une URL publique.

## 7 et 8. Rapport et présentation

Documentations techniques par module et rapport final, tous **générés par script**, puis support de présentation.

## Principes transversaux

### Les notebooks sont générés

Chaque notebook (**01\_eda.ipynb**, **01\_ml.ipynb**, **01\_data\_mining.ipynb**) est **produit par un script \_build\_\*.py**. Pour modifier un notebook, on édite le générateur puis on régénère et réexécute — on n'édite jamais le **.ipynb** à la main. Cela garantit que le code du notebook reste versionnable, relisible et reproductible.

### Les PDF sont générés

Les documentations techniques sont produites par des scripts qui **recalculent les statistiques depuis l'entrepôt** au moment de la génération. Aucun chiffre n'est saisi en dur : un rapport ne peut donc pas diverger des données qu'il décrit.

### Reproductibilité

Tout livrable doit pouvoir être reconstruit à partir du dépôt cloné et de **requirements.txt**. Le jeu de données source, l'entrepôt DuckDB et le rapport Power BI sont versionnés (données publiques), de sorte qu'un clone donne accès à l'ensemble sans étape manuelle.

### Séparation des responsabilités

Chaque module a une fonction claire et communique par des interfaces définies : la table analytique entre l'ETL et l'entrepôt, le fichier DuckDB entre l'entrepôt et ses consommateurs, le pipeline sérialisé entre le machine learning et l'application. Cette séparation rend chaque brique compréhensible et testable isolément.